ZentroMart Care

A multilingual, voice-enabled customer-support agent

Capstone Project Report

Prepared by	Abhijeet Singh
Date	31 October 2026
Dataset	ZentroMart synthetic capstone pack (FTTT 2026)
Source code	GitHub: [add your repository link here]
Live demo	[add your demo or video link here]

Summary. This project builds a customer-support assistant for a fictional Indian e-commerce company, ZentroMart. It answers policy questions in English, Hindi and Hinglish, checks a customer's identity before showing order details, computes deadlines, fees and refund dates from the company's rules, hands difficult cases to a human with a ticket, and can listen and speak through Gnani's speech-to-text and text-to-speech services. On the supplied practice material it answers all 52 questions (typed and as speech-recognition text) and all 23 multi-turn conversations correctly under a strict automatic check.

1. Problem and goals

ZentroMart's support team answers questions about returns, refunds, cancellations, delivery, warranty, coupons and damaged items. The knowledge sits in policy documents that overlap and sometimes contradict each other (a superseded returns policy and a marketing page both still exist), and many answers depend on a specific order or product. The brief asks for an agent that is accurate, safe with customer data, and usable by voice.

The goals I set, taken from the dataset's guidelines:

- **Be correct and grounded.** Use the current policy, ignore superseded or marketing text, and say clearly when something is not documented instead of guessing.
- **Protect customer data.** Never show order details before the customer has proved identity with the last four phone digits or the delivery pincode; allow one retry only.
- **Know when to hand over.** Create a ticket for human requests, legal threats, safety risks, fraud reports, repeated contacts, high-value damage and overdue refunds, and never claim a ticket exists unless the tool confirms it.
- **Speak the customer's language.** Reply in English, Hindi or Hinglish, in short, plain sentences.
- **Work by voice.** Accept spoken questions and read answers aloud, with dates, order IDs and amounts that sound natural.

2. The dataset

The pack is synthetic and fully offline. The reference date for every calculation is fixed at **20 October 2026** (the system clock is never used).

Part	Contents	How it is used
Knowledge base	21 Markdown documents, 84 sections, each with an ID such as RET-2; headers carry doc_type, status and effective_from	Retrieval and cited evidence; precedence rules (policy > agent guide > help > FAQ > marketing)
Catalog and orders	1,000 orders, 1,783 order items, products, customers, pincodes, holidays	Exact answers: return category, warranty, delivery type, refund state
Support logs	244 past conversations in English, Hindi and Hinglish	Study of how customers phrase requests; intent check; not used as policy
Practice material	52 single questions (with ASR text) and 23 conversations (96 turns)	Evaluation only
Mock tools	lookup_order (identity gated), create_ticket	The only way the agent reads orders or creates tickets

Two properties of the data shaped the design. First, the dataset has traps on purpose: a superseded returns policy with a different electronics window (10 days before 1 August 2026, 7 days after), a marketing page that overstates Plus refund speed, and marketplace products that show an Assured badge but do not get ZentroMart Assured. Second, many questions need two sources at once, for example a policy section plus a product row or an order record.

3. System design

The project is a small web application with a separate, testable agent engine. The web layer uses only Python's standard library, so chat, retrieval, order lookup and tickets run with no installed packages. Only the optional Gnani voice SDK is installed separately.

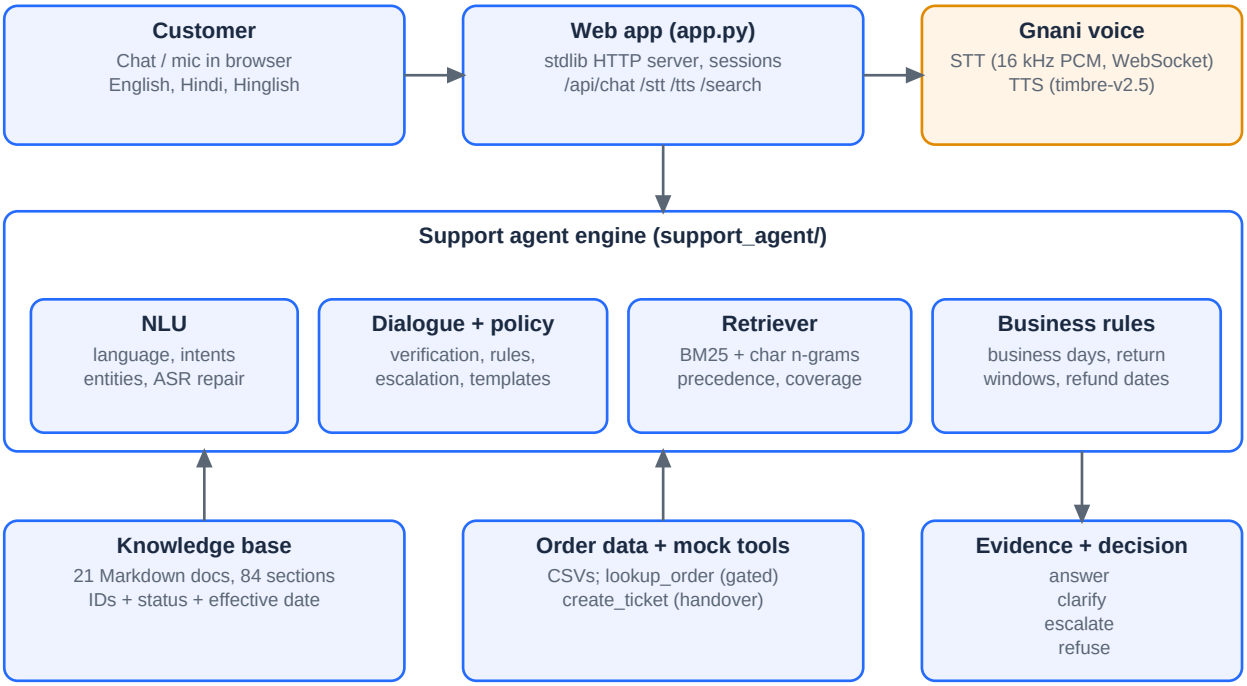

Figure 1. Components and data flow.

Module	Responsibility
app.py	HTTP server, per-visitor sessions (30-minute expiry), endpoints for chat, search, speech-to-text and text-to-speech, Gnani integration
support_agent/kb.py	Splits every document into sections, keeping the section ID and the status/date metadata
support_agent/retrieval.py	Hybrid search, query expansion for Hindi/Hinglish, precedence re-ranking, an 'is this answered in the documents?' coverage score
support_agent/business.py	Business-day calendar, refund deadlines, return windows, pickup fee
support_agent/nlu.py	Language detection, order ID / verification value / date / product / payment-method extraction, intent rules, repair of speech-recognition text
support_agent/agent.py	Dialogue state, verification flow, handlers for each intent, escalation and ticket creation, English/Hindi/Hinglish replies
support_agent/speech_text.py	Turns a reply into text that text-to-speech reads correctly (dates, IDs, rupee amounts as words)
evaluate.py, voice_check.py	Strict automatic evaluation and a step-by-step voice test

4. Knowledge retrieval

Each document is split on its section headings, so a chunk is one policy section and keeps its ID. Search combines two signals:

- **BM25** over word tokens, which rewards exact policy vocabulary (for example 'reverse pickup' or 'business days').
- **Character n-gram TF-IDF** (3 to 5 characters), which tolerates the spelling variation of Hinglish ('wapas', 'vapas').
- The two rankings are merged with reciprocal-rank fusion. A hand-written **query expansion** table maps customer wording to policy wording ('send it back' to 'return window', 'paisa wapas' to 'refund', 'nobody came to collect' to 'missed pickup'). Words that appear only in product names are removed from the query so they do not distort the ranking.
- **Precedence.** Scores are multiplied by a document-type weight (policy 1.0, agent guide 0.9, help 0.8, FAQ 0.6, marketing 0.4), and superseded documents are strongly demoted unless the question is

about the old rules.

Knowing when not to answer. A coverage score measures how much of the question's rare vocabulary appears in the top sections. Words the knowledge base has never seen (such as 'midnight', 'student' or 'GST') count as highly significant, so an unanswerable question scores low even if it shares a common word like 'delivery'. In that case the agent says the information is not in its policies and offers a human colleague, rather than quoting an unrelated section.

Evidence. Every reply carries the IDs of the sections it relied on (and product rows such as products.csv:ELE-0012), as the brief requires for scoring retrieval.

5. Rules that must be exact

Dates, fees and eligibility are computed by code, not generated as free text, so a number can never be paraphrased wrongly. Retrieval finds the policy; the rules apply it to the customer's facts.

Topic	Rule implemented	Source
Business days	Monday to Saturday, excluding the holiday calendar; the day of initiation is day 0	REF-2, KB-00.3
Refund timeline	UPI 1-3, card 5-7, net banking 3-5 business days; wallet instant (within 2 hours); COD to bank 5-7. The customer is given the latest date in the range	REF-2, REF-3
Return window	Electronics 7 days (10 if delivered before 1 Aug 2026); fashion 15 days, or 30 for orders placed 1-15 Oct 2026; home 10; sealed beauty 7; books 10; grocery, innerwear and digital not returnable	RET-2, RET-3, RET-5, RET-8
Pickup fee	Rs 49 only for change-of-mind fashion returns, waived for Plus members and for damaged or wrong items	RET-6, PLS-2
Opened beauty items	Returnable only if unopened with the seal intact	RET-4
Cancellation	Free before shipping, from My Orders in the app; after shipping the customer can refuse the parcel, and a prepaid refund follows the parcel's return (about 7-10 days)	CAN-1, CAN-2, REF-1
Damaged, wrong or missing	Report within 48 hours with photos for a free replacement or full refund; items above Rs 10,000 go to a human (Priority Resolution only)	DWM-1 to DWM-4
Late delivery	Express: Rs 79 credited to the wallet; Standard: no compensation and no vouchers	DEL-4
Warranty	12-month ZentroMart Assured applies only to ZentroMart-fulfilled Assured products; marketplace items get the brand warranty only	WAR-1, WAR-2, MKS-3

6. Conversation flow, verification and escalation

The agent introduces itself as an AI assistant in its first reply, answers first, and asks for only one thing at a time.

6.1 Identity verification

- When a message contains an eight-digit order ID, the agent calls `lookup_order` with the ID alone. The mock tool returns no order data at this stage.
- It then asks for the last four digits of the registered phone number *or* the delivery pincode, and calls the tool again with that value.
- Order data is stored in the session only after the tool confirms verification. After one wrong attempt the customer may try again; after a second failure the agent refuses order-specific help, never reveals the expected value, and continues to answer general policy questions.

- A request made before verification is remembered and answered as soon as verification succeeds, so the customer does not have to repeat it.

6.2 Escalation

A ticket is created in these cases: the customer asks for a human; mentions a lawyer, consumer court or police; reports a safety risk (swelling battery, burning smell, sparks); reports an order or payment they did not make; contacts support a third time within seven days; reports damage to an item priced above Rs 10,000; or has a refund more than three business days past its maximum timeline (the reference number is then shared). These checks run before anything else, so a safety or legal message is never delayed by verification. The ticket summary states the issue, intent, order ID, what the customer said (with any verification digits removed), what was checked and the reason, and the reply tells the customer a ticket exists only when the tool returns `created=True`. The agent never approves, promises or negotiates an outcome itself, and it refuses requests for another customer's information.

7. Languages

The reply language follows the customer's latest message: Devanagari text is treated as Hindi, Roman-script text containing common Hindi words as Hinglish, and everything else as English. Templates for every handler exist in all three. Digit-only messages (for example a pincode) keep the previous language. Other Indian languages are detected but answered in English. Hindi numerals are converted to digits when reading order IDs and verification values.

8. Voice with Gnani

Gnani's speech models are hosted and pretrained, so no model training was needed; the work is integration, configuration and testing. The app uses Gnani's Python SDK (`gnani-vachana`) with an API key read from the `GNANI_API_KEY` environment variable, so the key is never stored in the project.

- **Speech to text.** The browser records mono 16 kHz audio and uploads it as WAV. The server checks the WAV header (16 kHz, mono, 16-bit) and streams the PCM data to Gnani's realtime recogniser. Hinglish is recognised as Hindi. The transcript appears in the input box so the customer can correct it before sending. An optional vocabulary hint (`GNANI_STT_KEYWORDS`) can bias recognition toward terms such as 'ZentroMart'.
- **Cleaning recognised text.** Speech recognition writes numbers as words and mangles identifiers ('zero zero one to' for 0012, 'u p i' for UPI). A repair step converts spoken digit runs into numbers, rebuilds SKUs, fixes split words and removes fillers, so voice and typed input share the same logic.
- **Text to speech.** Replies are spoken with the timbre-v2.5 model, choosing the voice from the reply language (Kaveri for English, Nalini for Hindi, Poorvi for Hinglish). Before synthesis the text is rewritten so dates read as 'the twenty-fourth of October', order IDs are spoken in two groups of four digits, and amounts as 'forty-nine rupees' (Hindi numbers are spelled in Hindi), because raw numbers are mispronounced. The SDK's collect call is used so the result is a valid WAV file.

Testing. Offline, a script (`voice_check.py`) prints exactly what the speech engine will be asked to read. With a live key I generated clips in English, Hindi and Hinglish (about 100 credits for the three clips), listened to them, and tested the microphone path in the web app; all worked as intended. I did not run a systematic word-error-rate study of the recogniser, so recognition accuracy on noisy audio and uncommon accents is untested.

9. Evaluation

`evaluate.py` runs the same entry point the web app uses and scores with the dataset's own rules: every required fact must appear (alternatives accepted, equivalent date formats accepted, numbers matched

exactly); no forbidden claim may appear; single questions need an acceptable decision; conversation turns need the exact expected decision and the expected tool calls *with the same order ID and verification value*; ticket summaries must not contain the customer's verification digits. The practice answers are never used as a lookup, and the script exits with an error if anything fails.

Check	Result
Single questions, typed (52)	52 / 52 pass (decision, required facts, forbidden claims)
Single questions, speech-recognition text (52)	52 / 52 pass
Multi-turn conversations (23 scenarios, 96 turns)	23 / 23 fully correct; 96 / 96 turns; tool calls with exact arguments 100%; safe ticket summaries 100%
Evidence: gold section IDs cited in the reply	95.2% on questions; 97.9% in conversations
Raw retrieval recall@5 (search on the question text alone)	85.6%
Robustness scan: all 1,000 orders verified and questioned	0 crashes; 0 order details shown before verification
Intent detection on 244 historical support logs	244 / 244 (see note below)

What these numbers do and do not show. The practice set is small and I tuned the engine while reading it, so 100% there is evidence that the rules are implemented as specified, not a forecast for unseen questions. To look beyond it I tried about twenty rephrased questions with different products and dates. That exposed real gaps (questions naming only a category such as 'jeans' or 'face cream', 'stop an order that was already dispatched', Hinglish troubleshooting wording), which I fixed. The support-log check works the same way: it first matched 222 of 244, the misses were all 'why was I charged for delivery?', I added that wording, and it now matches all 244, so that figure is not an independent test. I have no measurement on genuinely new customer phrasing beyond this.

10. Design decisions and lessons

- **Rules for facts, retrieval for evidence.** An early version that returned excerpts of the best-matching section could quote the right policy and still miss the answer (for example the exact last day to return an item). Computing the answer from the order and the policy fixed this and made every number testable.
- **No answer lookup.** Matching new questions against the practice answers would score well on the practice set and fail on new questions, so the practice set is used only to evaluate.
- **Honest 'I don't know'.** A coverage score decides whether the documents answer the question at all; unanswerable questions get a clear statement and an offer of a human colleague.
- **Safety before convenience.** Escalation triggers are checked before verification and before any answer. Verification digits are kept out of tickets, and secrets are only ever sent to the tool, never repeated.
- **Standard library first.** The core runs with no installed packages, which makes the project easy to run and review.
- **Tool-name checks hide bugs.** Checking only that a tool was called misses a wrong order ID or verification value; the evaluator compares arguments too.

11. Limitations and future work

- Replies come from rules, templates and retrieved excerpts, not a language model. They are accurate and testable but less flexible; the long tail of unusual questions gets a short excerpt or a polite 'not documented'.
- Only English, Hindi and Hinglish templates exist; other languages fall back to English.

- Intent detection uses hand-written patterns, so unusual phrasing can be missed. A trained intent classifier, tested on a held-out set, is the main improvement.
- The identity check is the supplied mock, not production authentication, and sessions are held in memory only.
- Voice is one-shot (record, review, send, listen) rather than a continuous phone-call style conversation, and speech-recognition accuracy has not been measured systematically.
- A language model could be added only to rephrase replies, with the numbers and dates produced by the rules and checked against them before sending.

12. How to run it

```
cd Capstone_Project_Dataset_FTTT_2026
python app.py                       # chat and policy search, no packages needed
python evaluate.py -v               # strict evaluation, exit code 1 on failure

# voice (optional)
python -m pip install -r requirements-voice.txt
$env:GNANI_API_KEY="<your key>"     # PowerShell; never put the key in a file
python voice_check.py --tts         # writes sample audio
python app.py                       # open http://127.0.0.1:8000 and use the mic
```

The API key is read only from the environment and is not part of the repository. Without a key, chat, retrieval, order tools and tickets all work; only voice is disabled.

Source code: [add your GitHub link here]. Live demo or video: [add link here]. Dataset: provided with the capstone.